
Project 3: Memory-Mapped Files

Implementing mmap/munmap in xv6

Due date
2026. 06. 19. 23:59

Overview

- **Goal**

- Add **mmap** and **munmap** to xv6. Map a file into a process's address space. (Details on the next slide.)

- **Submission Guidelines**

- Submit the modified **xv6 source code** and the **report** for this assignment.
- Push all your source code and your report to the following GitHub Classroom repository's **main branch**:
 - https://classroom.github.com/a/g6u8SPR_

- **Deadline: June 19, 2026 (Fri) 23:59**

- Late submissions are allowed, but your final score will be reduced by **10%** for each **2-hour window** following the due date.
- Grading is based on your **final commit**. Be extremely cautious about committing after the deadline, as it will trigger the late penalty.

- **Short quiz** right before the final exam on **June 20 (Saturday)**

The big picture

- **mmap** maps a file into a process's address space, so the file looks like an array in memory.
 - No data is read up front. Pages are filled in **lazily** on first access, via a page fault.
- **munmap** removes a mapping. For MAP_SHARED, modified pages are written back to the file first.
- **What you will build:**
 - Per-process bookkeeping (VMAs), a lazy page-fault handler, and write-back on unmap, plus fork / exit handling.
 - The next slides cover each piece in detail.

What does mmap do?

```
int fd = open("bigfile", O_RDONLY);  
char *p = mmap(1024*1024*1024, PROT_READ, MAP_PRIVATE, fd, 0); // 1GB  
printf("%c\n", p[500000000]); // 500MB
```

- read() calls used: 0.
- Memory used: the pages you touch, not the whole file.
- p[500000000] really is the 500-millionth byte of the file.
- By the end of project, you will have built this.

The API for this lab

```
$ vim kernel/user.h
```

```
char *mmap(int len, int prot, int flags, int fd, int offset);  
int  munmap(void *addr, int len);
```

- **len** – size to map or unmap, a positive PGSIZE multiple.
 - **prot** – access permission: PROT_READ, PROT_WRITE, or both. Sets the PTE permission bits on the mapped pages.
 - **flags** – sharing mode, exactly one of MAP_SHARED (writes back to the file) or MAP_PRIVATE (changes stay process-local).
 - **fd** – the open file to map. fildup bumps its refcount so the mapping survives a later close of the fd.
 - **offset** – start position in the file to map.
 - **addr** – start address to unmap. No addr argument in mmap system call, the kernel picks the addr by it self.
-
- If there is any invalid argument returns -1.

The VMA: the kernel's bookkeeping

- For each mapping, the kernel remembers: which VA range, which file, which prot.
- A fixed array of 16 slots per process is en

```
$ vim kernel/proc.h
```

```
#define NVMA 16
```

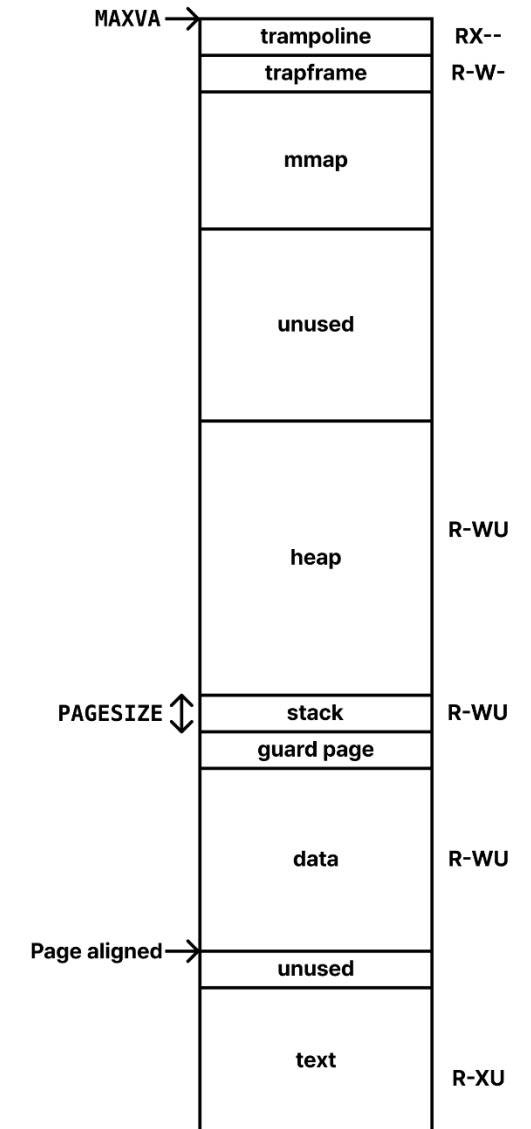
```
struct vma {  
    int used;           // slot in use?  
    uint64 addr;        // start VA (page-aligned)  
    uint64 length;      // bytes (PGSIZE multiple)  
    int prot;           // PROT_READ / PROT_WRITE  
    int flags;          // MAP_SHARED / MAP_PRIVATE  
    int offset;         // file offset (always 0)  
    struct file *f;     // refcount via filedup  
};
```

```
$ vim kernel/proc.h
```

```
struct proc {  
    // ... existing fields ...  
    struct vma vmas[NVMA];  
};
```

mmap

- Register VMA slot and address space.
 - mmap region sit below TRAPFRAME, growing downward.
 - Heap (p->sz) and mmap must not overlap.
 - mmap does not touch p->sz. sbrk does not touch mmap.
- Fill in the VMA.
- **mmap does not allocate the memory and read the file.**
It just records intent.



The trick: lazy page fault

- mmap does not read the file. It just records intent.
- First access raises a page fault.
- The handler reads the page that holds it (PGSIZE bytes) from the file and maps it.
- The instruction retries.
- Mapping a 1GB file allocates no physical pages until you touch one.

munmap and write-back

- `munmap(addr, len)` removes `[addr, addr + len)`.
- The range always sits at the start or end of a VMA, or covers the whole VMA.
 - Never a middle hole.
- For `MAP_SHARED`, modified pages must be written back to the file before being removed.
- When the whole VMA is gone, close the file and clean the VMA.

The skeleton: provided vs your work

- [Already wired up](#) so you don't touch plumbing:
 - Syscalls (syscall.{h,c}, usys.pl, user.h, Makefile): mmap / munmap reach the kernel.
 - Data structures (proc.h): NVMA, struct vma, vmas[NVMA] on proc. allocproc zeros slots.
 - Trap path (trap.c): usertrap calls vmfault (lazy heap) then mmapfault (your mmap regions), kills if both decline.
 - Stubs in kernel/sysfile.c: sys_mmap, sys_munmap, mmapfault all return -1.
- You replace the bodies of those three stubs (sys_mmap, sys_munmap, mmapfault) and add fork/exit hooks in kernel/proc.c
- Almost all inside kernel/sysfile.c.
- Do not modify user/mmaptest.c.

Your implementation: four steps

1. Implement `sys_mmap`.
 2. Implement `mmapfault` body.
 3. Implement `sys_munmap` with `MAP_SHARED` write-back.
 4. Hook `kfork` / `kexit`.
- **Verification:** the project is complete once `mmaptest` and `usertests -q` both pass.
 - Build and run both after every step to catch regressions early.

Step 1. `sys_mmap`

- The VMA struct, the array on struct proc, and the empty slot initialization are already provided.
- You implement the body of `sys_mmap` in `kernel/sysfile.c`:
 1. validate args
 2. find a free VMA slot
 3. pick an unused VA below TRAPFRAME, above `p->sz`
 4. fill in the VMA and return its addr. Increment ref count of file using `filedup(f)` when you fill the file filed in the VMA.
- No physical pages yet.
 - Done when the first `mmap` succeeds, but the first access page-faults and kills the test.

Step 2. mmapfault body

- Replace the stub of `mmapfault(va)` in `kernel/sysfile.c`:
 1. Find the VMA covering `va`. (Return -1 if not found).
 2. Allocate a physical page and initialize it with zeros.
 3. Read the file data into the allocated page.
(hint: Refer to the inode reading logic inside `fileread()`)
 4. Translate `prot` to PTE permission bits, and map the physical page to the `va`.
 5. Return the physical address mapped to `va`.
- Short reads near EOF are fine. The rest of the page stays zero.
- Done when all tests through "test mmap two files" pass.

Step 3. munmap with write-back

1. Write back the data to the file if VMA is MAP_SHARED and free the physical page and clear the PTE.
 - If there is no PTE in the area (pages never faulted before) just skip the page.
 2. Adjust the VMA's addr, offset, length.
 3. Close the file and free the VMA slot if whole VMA gone.
- Reject middle-hole unmmaps with returning -1.
 - (Optional) Use the PTE_D (dirty) bit to skip clean pages; writing every present page also works.
 - Done when all single-process tests pass. "test fork" still fails.

Step 4. kfork and kexit

- kfork: child must see the same mappings as the parent.
 - copy the VMA array
 - do not share physical pages. The child will fault them in lazily.
- kexit: MAP_SHARED modifications must reach the file.
 - for each used VMA, run the unmap-with-writeback path.
 - execute this logic before the pagetable is freed.
- Done when
 - mmaptest: all tests succeeded
 - usertests -q reports ALL TESTS PASSED

Pitfalls to avoid

- `PROT_*` are not `PTE_*` bits. Translate explicitly:

```
int perm = PTE_U;  
if (v->prot & PROT_READ) perm |= PTE_R;  
if (v->prot & PROT_WRITE) perm |= PTE_W;
```

- `filedup` is mandatory. Without it, closing the `fd` dangles the file.
- No `addr` argument in the simplified signature. Use `argint / argfd` for the 5 arguments accordingly.
- Second fault on the same page is a permission violation. Return `-1` instead of calling `mappages` again (would panic with `mappages: remap`).
- `munmap` in `kexit` still needs `begin_op / end_op` to call `writei`.
- Run `usertests -q` after every step. `kfork` and `kexit` regress most easily.

Report

- **Design**

- Explain your implementation plan to meet the assignment requirements.

- **Implementation**

- Detail the specific modifications and additions you made to the given xv6 codebase. Clearly highlight how they differ from the original code and the purpose of these changes.

- **Results**

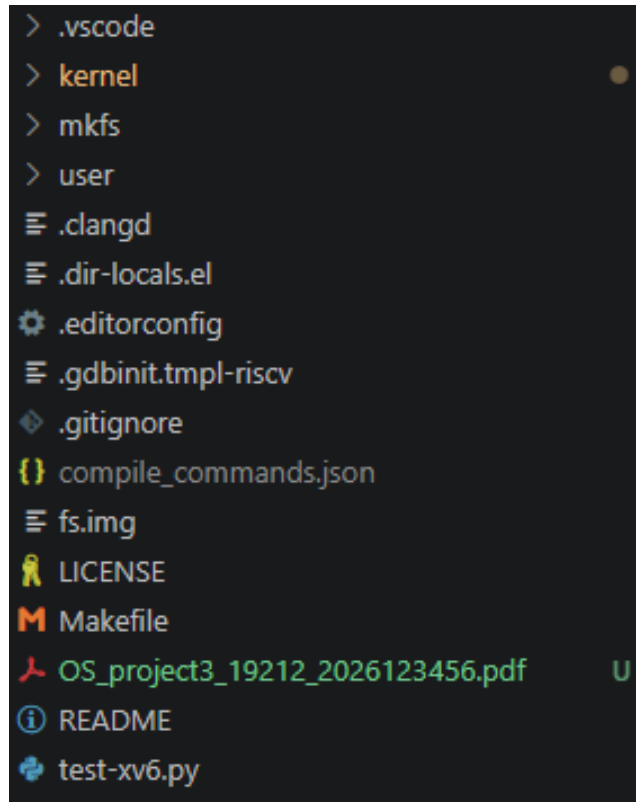
- Document the compilation and execution process. Include screenshots verifying that all requirements are met, alongside explanations of the execution flow.

- **Troubleshooting**

- If you encountered any issues during the assignment, describe the issues and your resolution process. For unresolved issues, explain what they were and what approaches you attempted.

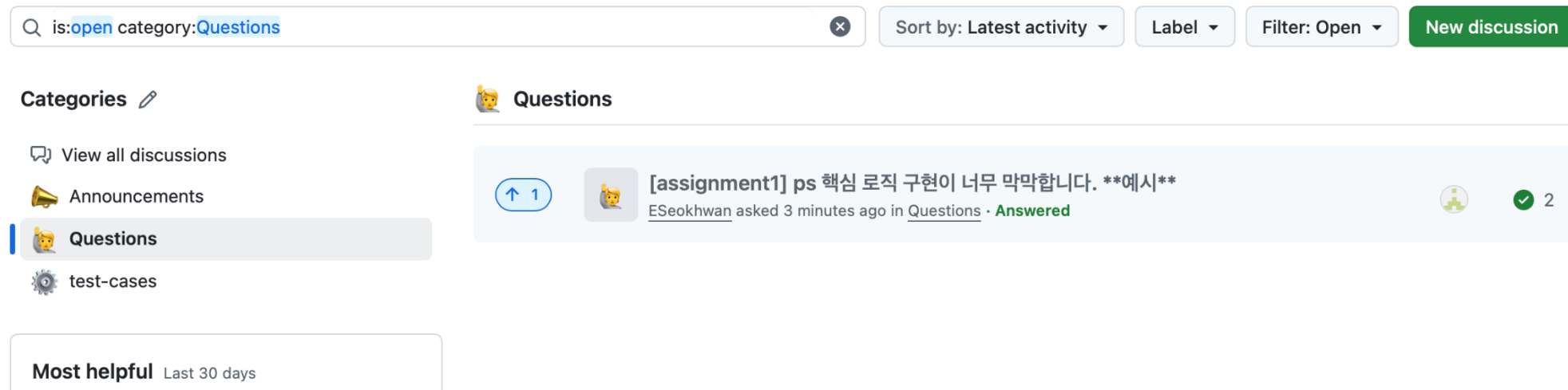
Report

- The report must be a PDF file strictly named: **OS_project3_[class number]_[student ID].pdf**
- You must commit and push your report PDF to your GitHub Classroom repository along with your xv6 source code.
- Make sure that report PDF is in the root directory of your repository, as shown in below.



Q&A

- For assignment related questions, please use the **Discussions tab** of xv6-sandbox repository.
 - <https://github.com/2026-HYU-ELE3021/xv6-riscv-sandbox/discussions>
- **Questions sent by email will not be answered.**
- For non-assignment questions or personal matters, please use an email like before.



- Students are highly **encouraged to answer each other's questions!** Active participation is greatly appreciated and helpful for everyone

Test Case Contribution (Optional)

- You can share your custom test cases on the **Discussions tab** of xv6-sandbox repository too.
 - <https://github.com/2026-HYU-ELE3021/xv6-riscv-sandbox/discussions>
- A structured template is already provided, so we highly encourage your active contributions.
- **This time, passing the provided test cases is all that's required for grading.**

